

Descrição Sumária

O programa implementa um sistema de gestão hospitalar que permite gerir enfermarias, episódios de internamento e analisar métricas de ocupação de camas. O sistema é composto pelas seguintes classes e funcionalidades:

Interface Analisavel — Define o contrato de análise de ocupação. Qualquer classe que implemente esta interface é obrigada a disponibilizar o cálculo de ocupação absoluta, taxa percentual e deteção de pressão (taxa superior a 85%), todos referentes a uma data específica.

```
1 package modelo;
2
3 import java.time.LocalDate;
4
5 public interface Analisavel { 1 usage 4 implementations new *
6
7     int getOcupacaoAbsoluta(LocalDate data); 2 usages 1 implementation new *
8
9     double getTaxaOcupacao(LocalDate data); 7 usages 1 implementation new *
10
11     boolean emPressao(LocalDate data); 4 usages 1 implementation new *
12 }
```

Classe Episodio — Construtores A classe mantém um contador estático para gerar identificadores únicos (EP1, EP2, ...). Dispõe de dois construtores: um simplificado para criação rápida e outro completo para reconstituir episódios a partir de ficheiros CSV, aceitando estado e data de alta diretamente.

```
38 public Episodio(String camaId, LocalDate dataAdmissao) { no usages new *
39     this(episodioid: "EP" + proximoNumero++, camaId, dataAdmissao, dataAlta: null, estado: "ATIVO");
40 }
41
42 public Episodio(String episodioId, String camaId, LocalDate dataAdmissao, LocalDate dataAlta, String estado) {
43     this.episodioid = episodioId;
44     this.camaId = camaId;
45     this.dataAdmissao = dataAdmissao;
46     this.dataAlta = dataAlta;
47     this.estado = estado == null || estado.isBlank() ? "ATIVO" : estado.trim().toUpperCase();
48     if (temAlta()) {
49         this.estado = "ALTA";
50     }
51 }
52 }
```

Análise temporal do Episódio — O método *estaAtivoEm* verifica se um episódio estava ativo numa data específica, testando se a data é posterior à admissão e anterior à alta (ou sem alta). O método *calcularLoS* calcula o número de dias de internamento, devolvendo -1 se o episódio ainda está ativo.

```
131     public boolean estaAtivoEm(LocalDate data) { 1usage new *
132         if (data == null || dataAdmissao == null) {
133             return false;
134         }
135         boolean depoisDaAdmissao = !data.isBefore(dataAdmissao);
136         boolean antesOuNaAlta = dataAlta == null || !data.isAfter(dataAlta);
137         return depoisDaAdmissao && antesOuNaAlta;
138     }
```

```
157     public long calcularLoS() { 1usage new *
158         if (!temAlta()) {
159             return -1;
160         }
161         return ChronoUnit.DAYS.between(dataAdmissao, dataAlta);
162     }
```

Classe Enfermaria — Adição de episódios com validação Antes de adicionar um episódio, o sistema valida se o *camald* é válido e se não existe conflito de datas para essa cama. Se passar nas verificações, o episódio é associado à enfermaria e adicionado à lista.

```
74     public boolean adicionarEpisodio(Episodio episodio) { 1usage new *
75         if (episodio == null || episodio.getCamaId() == null || episodio.getCamaId().isBlank()) {
76             return false;
77         }
78         if (existeConflitoDeCama(episodio)) {
79             return false;
80         }
81         episodio.associarEnfermaria(identificador);
82         episodios.add(episodio);
83         return true;
84     }
```

Verificação de conflito de cama — O método *existeConflitoDeCama* percorre todos os episódios existentes e verifica se algum ocupa a mesma cama num período sobreposto. Dois episódios sobrepõem-se se o início de um é anterior ao fim do outro e vice-versa.

```
208     public boolean existeConflitoDeCama(Episodio novoEpisodio) { 1usage new *
209         for (Episodio episodioExistente : episodios) {
210             if (episodioExistente.getCamaId().equalsIgnoreCase(novoEpisodio.getCamaId())
211                 && episodiosSobrepostos(episodioExistente, novoEpisodio)) {
212                 return true;
213             }
214         }
215         return false;
216     }
```

```

239 @ private boolean episodiosSobrepostos(Episodio primeiro, Episodio segundo) { 1usage new *
240     LocalDate inicioPrimeiro = primeiro.getDataAdmissao();
241     LocalDate fimPrimeiro = primeiro.temAlta() ? primeiro.getDataAlta() : LocalDate.MAX;
242     LocalDate inicioSegundo = segundo.getDataAdmissao();
243     LocalDate fimSegundo = segundo.temAlta() ? segundo.getDataAlta() : LocalDate.MAX;
244
245     return !inicioPrimeiro.isAfter(fimSegundo) && !inicioSegundo.isAfter(fimPrimeiro);
246 }
247 }

```

Análise de pressão por intervalo — O método *getPercentagemDiasEmPressao* itera dia a dia sobre o intervalo fornecido, verificando para cada data se a enfermaria está em pressão, e devolve a percentagem de dias em que isso ocorreu.

```

185 public double getPercentagemDiasEmPressao(LocalDate dataInicio, LocalDate dataFim) {
186     if (dataInicio == null || dataFim == null || dataInicio.isAfter(dataFim)) {
187         return 0.0;
188     }
189     int totalDias = 0;
190     int diasEmPressao = 0;
191     LocalDate dataAtual = dataInicio;
192     while (!dataAtual.isAfter(dataFim)) {
193         totalDias++;
194         if (emPressao(dataAtual)) {
195             diasEmPressao++;
196         }
197         dataAtual = dataAtual.plusDays( daysToAdd: 1);
198     }
199     return totalDias == 0 ? 0.0 : (diasEmPressao * 100.0) / totalDias;
200 }

```

Classe Hospital — O hospital gere as enfermarias com validação de duplicados e pesquisa insensível a maiúsculas. O método *listarEnfermariasOrdenadasPorTaxaOcupacao* ordena as enfermarias por taxa decrescente e, em caso de empate, por identificador.

```

44 public boolean adicionarEnfermaria(Enfermaria enfermaria) { 3usages new *
45     if (enfermaria == null || obterEnfermaria(enfermaria.getIdentificador()) != null) {
46         return false;
47     }
48     enfermarias.add(enfermaria);
49     return true;
50 }

```

```

76 public List<Enfermaria> listarEnfermariasOrdenadasPorTaxaOcupacao(LocalDate data) { 1usage new *
77     List<Enfermaria> ordenadas = getEnfermarias();
78     ordenadas.sort(Comparator.comparingDouble((Enfermaria e) -> e.getTaxaOcupacao(data)).reversed()
79         .thenComparing(Enfermaria::getIdentificador));
80     return ordenadas;
81 }

```

Classe AnalisadorEstatistico — **Cálculo de LoS** O método *calcularEstatisticasLoS* percorre os valores de LoS dos episódios com alta e calcula a média, desvio padrão, mínimo e máximo, devolvendo um objeto *SumarioLoS*. O método *ordenarPorTaxaOcupacao* usa uma classe anónima como Comparator para ordenar as enfermarias.

```

116 @ public static SumarioLoS calcularEstatisticasLoS(Enfermaria enfermaria) { 1 usage new *
117     List<Long> valores = enfermaria.getValoresLoS();
118     if (valores.isEmpty()) {
119         return new SumarioLoS( totalEpisodios: 0, media: 0.0, desvioPadrao: 0.0, minimo: 0, maximo: 0);
120     }
121
122     long minimo = valores.get(0);
123     long maximo = valores.get(0);
124     double soma = 0.0;
125     for (Long valor : valores) {
126         soma += valor;
127         if (valor < minimo) {
128             minimo = valor;
129         }
130         if (valor > maximo) {
131             maximo = valor;
132         }
133     }
134
135     double media = soma / valores.size();
136     double somaQuadrados = 0.0;
137     for (Long valor : valores) {
138         double diferenca = valor - media;
139         somaQuadrados += diferenca * diferenca;
140     }
141     double desvioPadrao = Math.sqrt(somaQuadrados / valores.size());
142
143     return new SumarioLoS(valores.size(), media, desvioPadrao, minimo, maximo);
144 }

```

```

171 @ public static void ordenarPorTaxaOcupacao(List<Enfermaria> enfermarias, LocalDate data) { no usages new *
172     enfermarias.sort(new Comparator<Enfermaria>() { new *
173         @Override new *
174         public int compare(Enfermaria primeira, Enfermaria segunda) {
175             int comparacao = Double.compare(segunda.getTaxaOcupacao(data), primeira.getTaxaOcupacao(data));
176             if (comparacao != 0) {
177                 return comparacao;
178             }
179             return primeira.getIdificador().compareToIgnoreCase(segunda.getIdificador());
180         }
181     });
182 }
183 }

```

Classe GestorFicheiros — Validação de dados Todos os campos dos CSV são validados antes de serem usados. O método *validarData* verifica o formato AAAA-MM-DD com mês e dia dentro de intervalos válidos. Entradas inválidas são registadas no ficheiro `erros_validacao.log`.

```

111 public static boolean validarData(String valor) { 3 usages new *
112     if (!validarString(valor)) return false;
113     String[] partes = valor.trim().split(regex: "-");
114     if (partes.length != 3) return false;
115     if (!validarInteiro(partes[0]) || !validarInteiro(partes[1]) || !validarInteiro(partes[2])) return false;
116     int mes = Integer.parseInt(partes[1]);
117     int dia = Integer.parseInt(partes[2]);
118     return mes >= 1 && mes <= 12 && dia >= 1 && dia <= 31;
119 }

```


Carregamento de episódios a partir de CSV — O método *processarLinhaEpisodio* valida cada linha do ficheiro CSV, verifica a existência da enfermaria, cria o episódio e regista a alta se presente. Em caso de conflito de cama, o episódio é rejeitado e o erro registado no log.

```

204 @ private static void processarEnfermariaGeral(String[] d, int linha, Usage new *
205                                     String id, int cap, Hospital h)
206     throws IOException {
207
208     if (d.length < 5 || !validarInteiro(d[3].trim())) {
209         logErro( mensagem: "Linha " + linha + ": GERAL requer acompanhantes e horario validos.");
210     } else {
211         int acomp = Integer.parseInt(d[3].trim());
212         if (acomp < 0) {
213             logErro( mensagem: "Linha " + linha + ": numero de acompanhantes nao pode ser negativo.");
214         } else if (!validarString(d[4])) {
215             logErro( mensagem: "Linha " + linha + ": horario de visitas em branco.");
216         } else {
217             EnfermariaGeral eg = new EnfermariaGeral(id, cap, acomp, d[4].trim());
218             for (int i = 5; i < d.length; i++) {
219                 if (validarString(d[i])) {
220                     eg.adicionarRecurso(d[i].trim());
221                 }
222             }
223             h.adicionarEnfermaria(eg);
224         }
225     }
226 }

```

Classe MainHospital — Ponto de entrada da aplicação. Solicita interativamente o diretório dos CSV, a data de referência e o intervalo de análise. Após carregar os dados com sucesso, a aplicação apresenta os resultados divididos pelos quatro requisitos principais:

Leitura de Dados e Estrutura (RF1): O sistema valida os inputs do utilizador, carrega os ficheiros através do GestorFicheiros e imprime a estrutura base do Hospital para garantir que os dados estão em memória sem erros de execução.

```

43     String directorioCsv = lerDirectorioCsv(leitor);
44     LocalDate dataReferencia = lerData(leitor, mensagem: "Introduza a data de referencia (AAAA-MM-DD): ");
45     LocalDate dataInicio = lerData(leitor, mensagem: "Introduza a data inicial do intervalo (AAAA-MM-DD): ");
46     LocalDate dataFim = lerDataFim(leitor, dataInicio);
47
48     Hospital hospital = new Hospital( nome: "Hospital Central XYZ");
49     GestorFicheiros.limparLog();
50
51     System.out.println("A carregar enfermarias do ficheiro CSV...");
52     GestorFicheiros.carregarEnfermarias(construirCaminho(directorioCsv, nomeFicheiro: "enfermarias.csv"), hospital);
53     System.out.println("    Enfermarias carregadas: " + hospital.getEnfermarias().size());
54
55     System.out.println(" A carregar episodios do ficheiro CSV...");
56     GestorFicheiros.carregarEpisodios(construirCaminho(directorioCsv, nomeFicheiro: "episodios.csv"), hospital);
57     System.out.println("    Consulte 'erros_validacao.log' para entradas rejeitadas.");
58
59     System.out.println("\n" + SEPARADOR);
60     System.out.println(" Estrutura carregada");
61     System.out.println(SEPARADOR);
62     System.out.println(hospital);
63     for (Enfermaria enf : hospital.getEnfermarias()) {
64         System.out.println("    " + enf);
65     }

```

Indicadores de Ocupação e LoS (RF2): Demonstração do cálculo da ocupação absoluta e da taxa percentual para uma data específica, seguida do sumário estatístico do Length of Stay (LoS) calculado para cada enfermaria.

```
67 // RF2 - INDICADORES DE OCUPAÇÃO
68 System.out.println("\n" + SEPARADOR);
69 System.out.printf(" Indicadores de Ocupação em %s\n", dataReferencia);
70 System.out.println(SEPARADOR);
71
72 for (Enfermaria enf : hospital.getEnfermarias()) {
73     System.out.printf("%n Enfermaria : %s\n", enf.getIdentificador());
74     System.out.printf(" Ocupação : %d / %d camas\n",
75         enf.getOcupacaoAbsoluta(dataReferencia), enf.getNumeroCamas());
76     System.out.printf(" Taxa : %.1f%%\n",
77         enf.getTaxaOcupacao(dataReferencia));
78     System.out.printf(" Estado : %s\n",
79         enf.emPressao(dataReferencia) ? "Em pressão" : "Estado normal");
80 }
81
82 System.out.println("\n" + SEPARADOR);
83 System.out.println(" Sumário de Length of Stay (LoS) por Enfermaria");
84 System.out.println(SEPARADOR);
85
86 for (Enfermaria enf : hospital.getEnfermarias()) {
87     AnalisadorEstatistico.SumarioLoS sumario = AnalisadorEstatistico.calculasEstatisticaLoS(enf);
88     System.out.printf(" %s: %s\n", enf.getIdentificador(), sumario);
89 }
```

Análise de Pressão (RF3): Apresentação da avaliação contínua da pressão do hospital, listando a percentagem de dias em que cada enfermaria ultrapassou os 85% de ocupação no intervalo de datas fornecido.

```
91 // RF3 - ANÁLISE DE PRESSÃO POR INTERVALO DE DATAS
92 System.out.println("\n" + SEPARADOR);
93 System.out.printf("Análise de Pressão [%s a %s]\n", dataInicio, dataFim);
94 System.out.println(SEPARADOR);
95
96 for (Enfermaria enf : hospital.getEnfermarias()) {
97     System.out.printf("%nEnfermaria %s:\n", enf.getIdentificador());
98     AnalisadorEstatistico.analisarPressaoPorIntervalo(enf, dataInicio, dataFim);
99 }
```

Pesquisa e Ordenação (RF4): Validação dos algoritmos baseados em Comparador, listando primeiro os episódios de uma enfermaria específica ordenados por admissão, e por fim, o ranking das enfermarias ordenadas por taxa de ocupação decrescente.

```
101      System.out.println("\n" + SEPARADOR);
102      System.out.printf("Episodios da Enfermaria %s %n", ID_ENFERMARIA_EXEMPLO);
103      System.out.println(SEPARADOR);
104
105      Enfermaria enfermarioExemplo = hospital.obterEnfermaria(ID_ENFERMARIA_EXEMPLO);
106      if (enfermarioExemplo != null) {
107          List<Episodio> episodiosOrdenados = enfermarioExemplo.getEpisodiosOrdenadosPorAdmissao();
108          for (Episodio ep : episodiosOrdenados) {
109              System.out.println("  " + ep);
110          }
111      }
112
113      System.out.println("\n" + SEPARADOR);
114      System.out.printf("Enfermarias por Taxa de Ocupacao em %s%n", dataReferencia);
115      System.out.println(SEPARADOR);
116
117      List<Enfermaria> enfermariasOrdenadas = hospital.listarEnfermariasOrdenadasPorTaxaOcupacao(dataReferencia);
118      for (Enfermaria enf : enfermariasOrdenadas) {
119          System.out.printf("  %-6s | Taxa: %5.1f%% | %s%n",
120                          enf.getIdentificador(),
121                          enf.getTaxaOcupacao(dataReferencia),
122                          enf.emPressao(dataReferencia) ? "Em pressao" : "Estado normal");
123      }
```

Diagrama de Classes no formato UML:

